



University
of Glasgow

W3-06: Spark Architecture

COMPSCI4064 (H) and COMPSCI5088 (M)

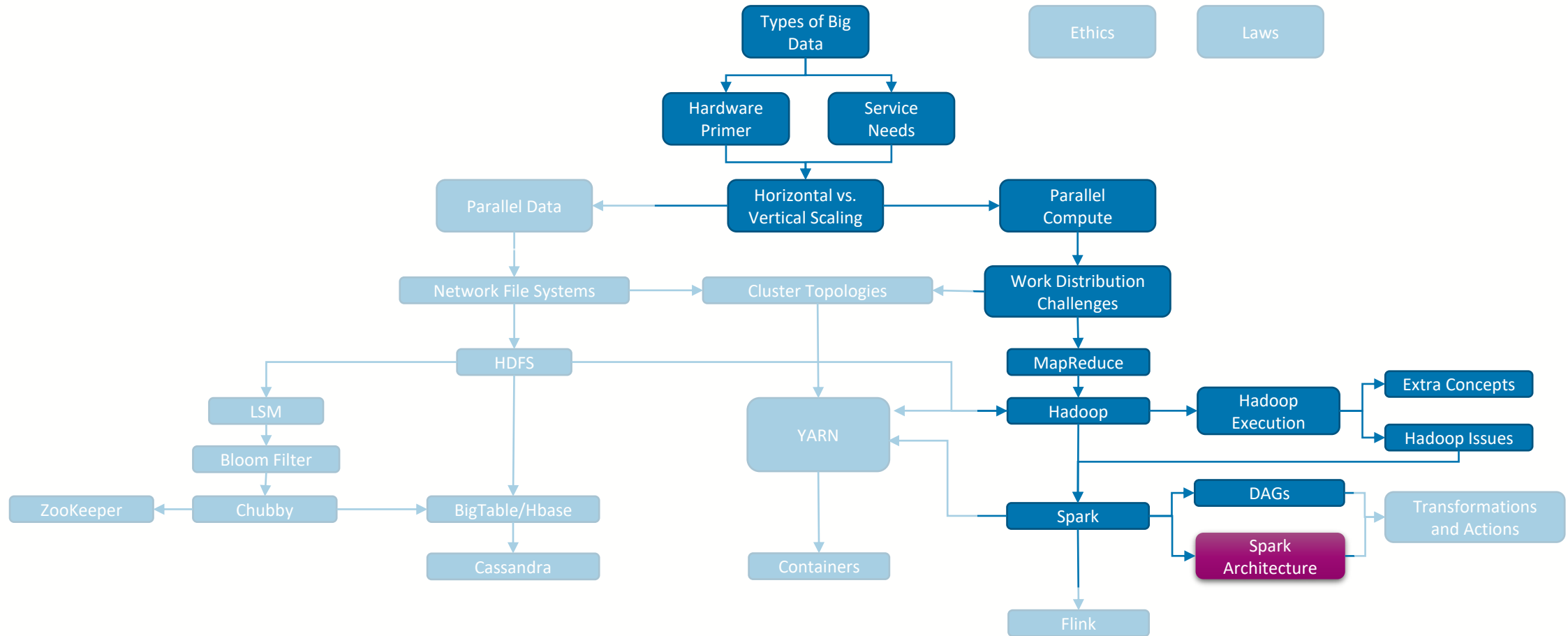
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



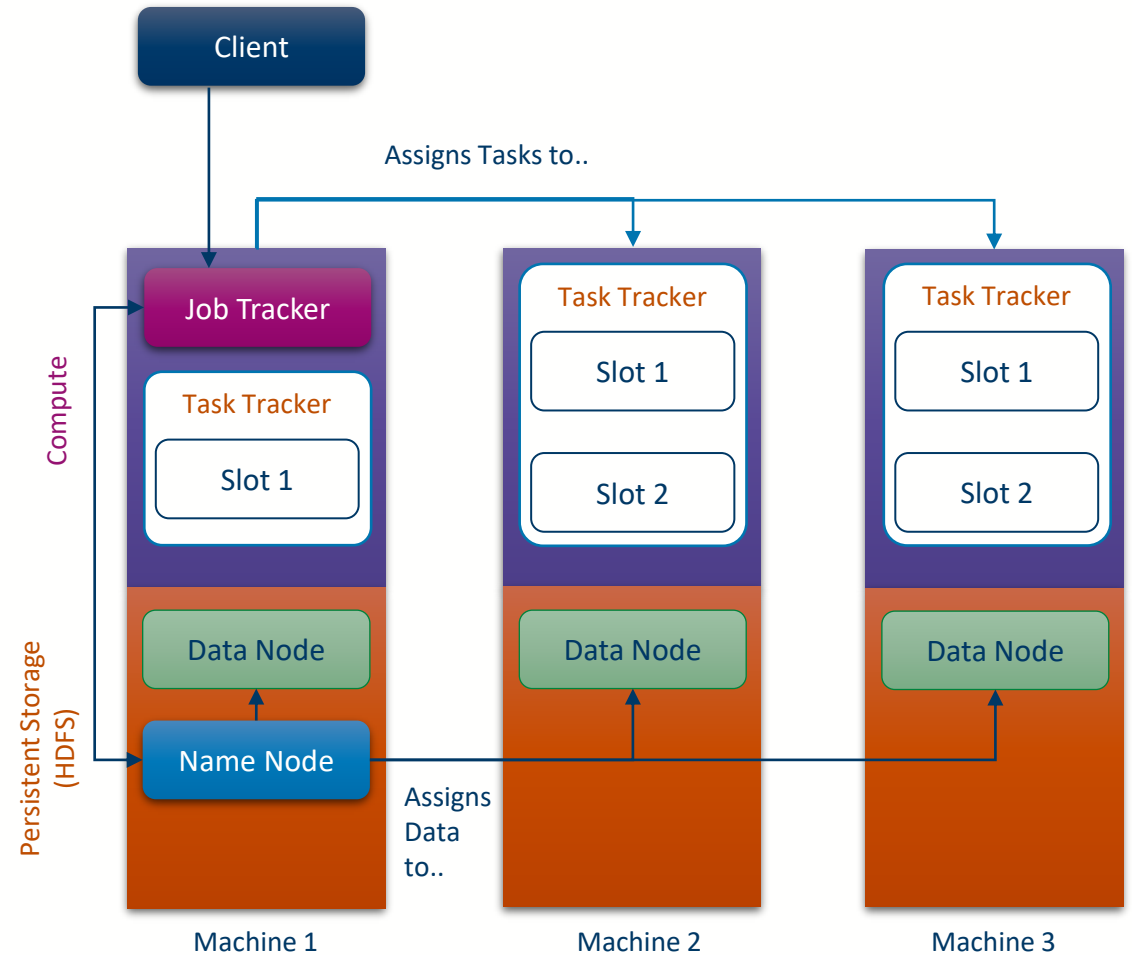
Where are we?





Recall the Hadoop Architecture...

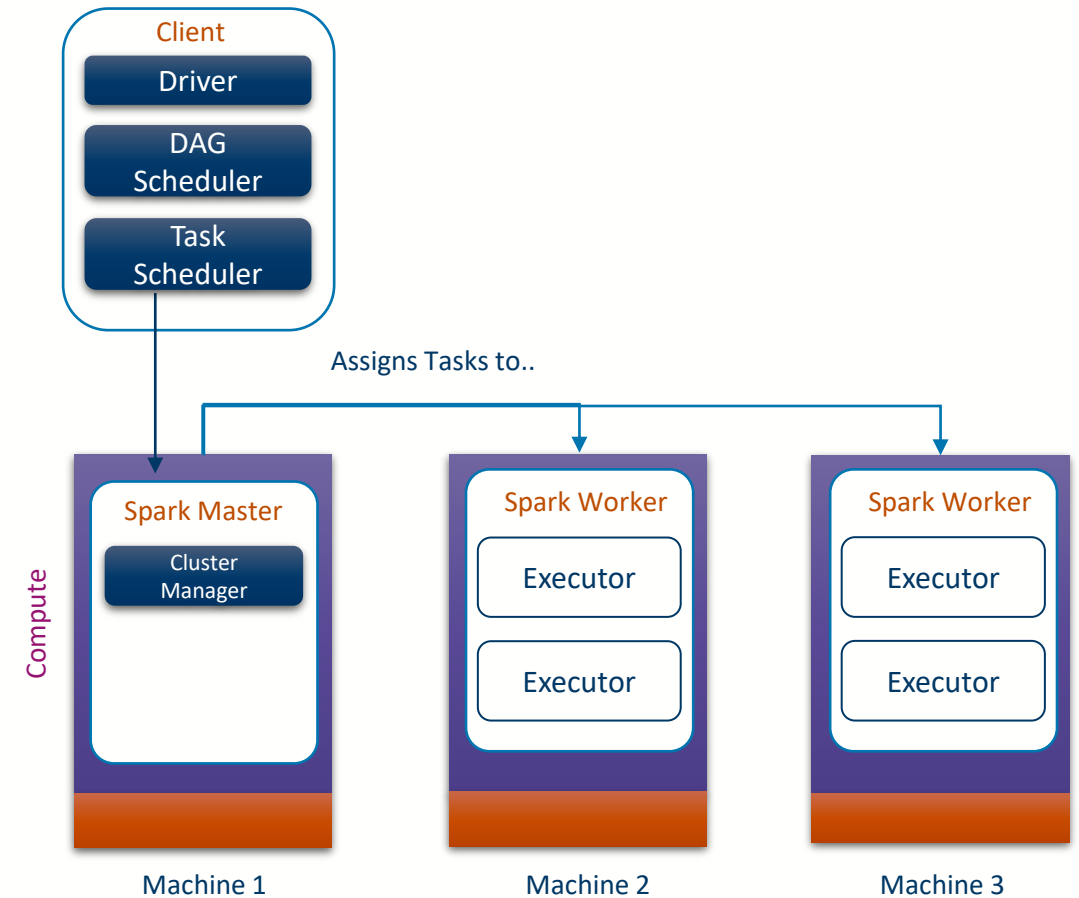
- Hadoop **Compute**
 - **Client**: Software that the user uses to submit work to the Hadoop cluster, communicates with the Job Tracker
 - **Job Tracker**: Receives work from the client, uses the Name Node to identify where data is located, assigns work to task trackers
 - **Task Tracker**: Executes Map and Reduce tasks
- Hadoop **HDFS**
 - **Name Node**: Tracks the location of all data held on the HDFS, handles replication
 - **Data Node**: Handles storage and access of the local files on the machine





Spark Architecture

- Spark **Compute**
 - **Client**: Software that the user uses to submit work to the Spark cluster, communicates with the Spark Master
 - **Driver**: The program that executes the Spark Job, may run on the Client or on a node in the cluster
 - **Spark Master**: Receives work from the client, analyses the DAG assigns work to task trackers, communicates with the Driver
 - **Spark Worker**: Manages the local execution of Transformations and Actions
 - **Executor**: A JVM that executes tasks using a ThreadPool
 - Responsible for executing application code for a specific task/stage, caching datasets/results





Important Differences

- No defined persistent storage layer
 - Spark may use temporary storage if RAM is exhausted
 - Application code defines how data is loaded
- The Driver program must remain alive until the Spark Job finishes (since it will be the subject of the final Action)
- The Driver does not need to be run on the Client
- Each Job gets its own Executer on the Workers (to avoid Jobs sharing memory), an Executer can run multiple tasks in parallel
 - Much faster to get a job going than in Hadoop



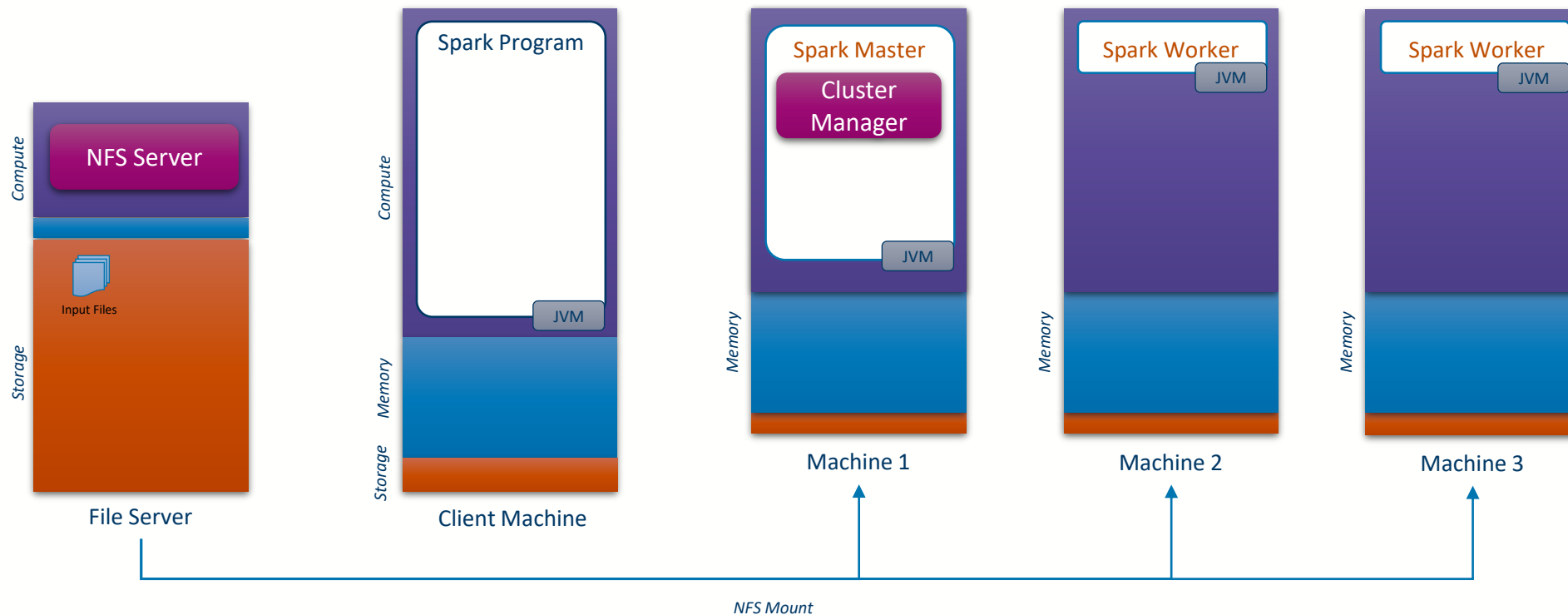
Spark Execution Phases

1. Client instantiates a **SparkContext** object (SC)
 - **Responsible for I/O** with the user, cluster manager service, and executors
 - **Initialises the Driver**, which registers the application with the Cluster Manager and gets a list of Executors
 - The Driver then is responsible for the Spark job
2. Client uses SC to **create input RDDs**
 - Partitioned/parallelised across cluster nodes
3. Client uses **Transformations on the input/intermediate RDDs**
 - Sequence of transformations creates a DAG of ops
 - Lazy evaluation → No computation takes place so far on the cluster
4. Client uses an **Action** to print/store results
5. **Action triggers submission of ops DAG** to the DAGScheduler within the Driver
6. The **DAGScheduler builds stages of tasks**
 - Defines what will be executed where
 - Implemented as an event queue
 - Groups together tasks based on their dependencies
 - Submits the execution plan to the TaskScheduler within the Driver
 - Resubmits failed stages, if their output was lost
7. The TaskScheduler:
 - **Ships tasks to Executors** (serialised RDD lineage + transformations)
 - **Monitors the execution/progress** of their computation
 - Launches tasks on Executors, according to resource and locality constraints
 - Decides where to run each task
 - Deals with stragglers (speculative execution)
 - **Returns the result to the DAGScheduler** (as a series of events)



Job execution anatomy

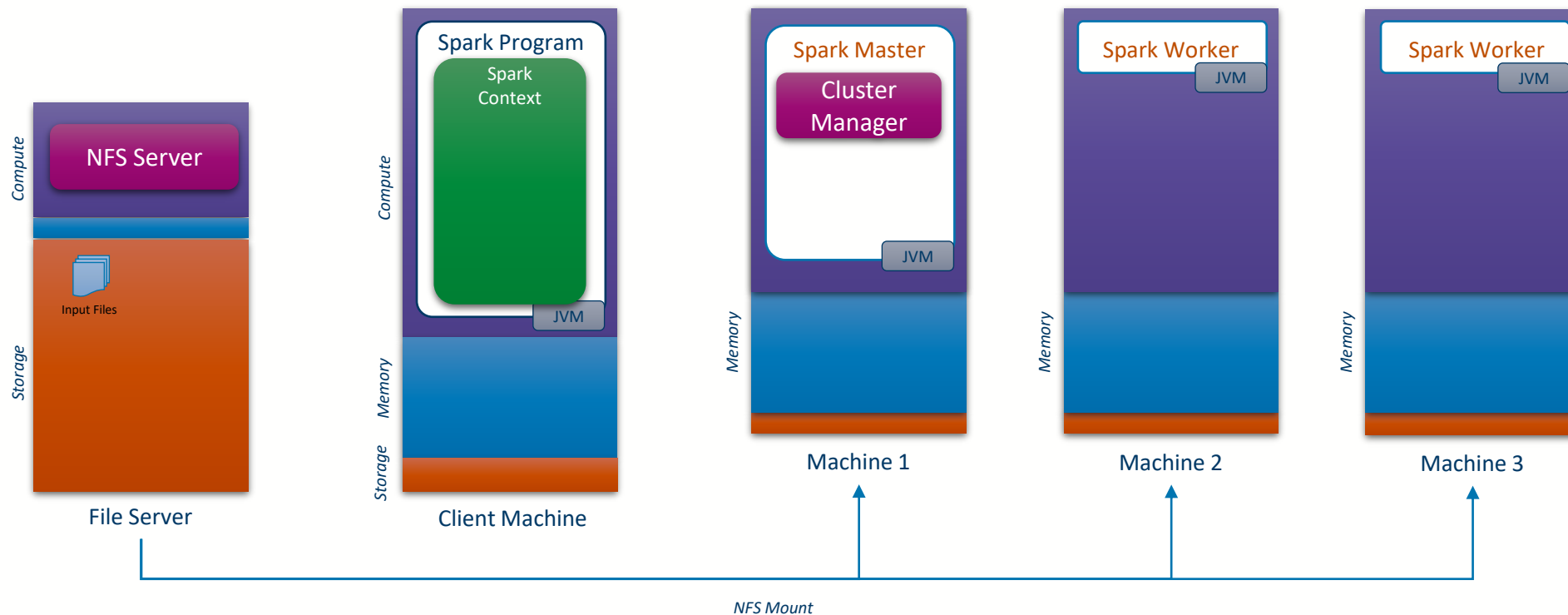
1 Run Spark Program





Job execution anatomy

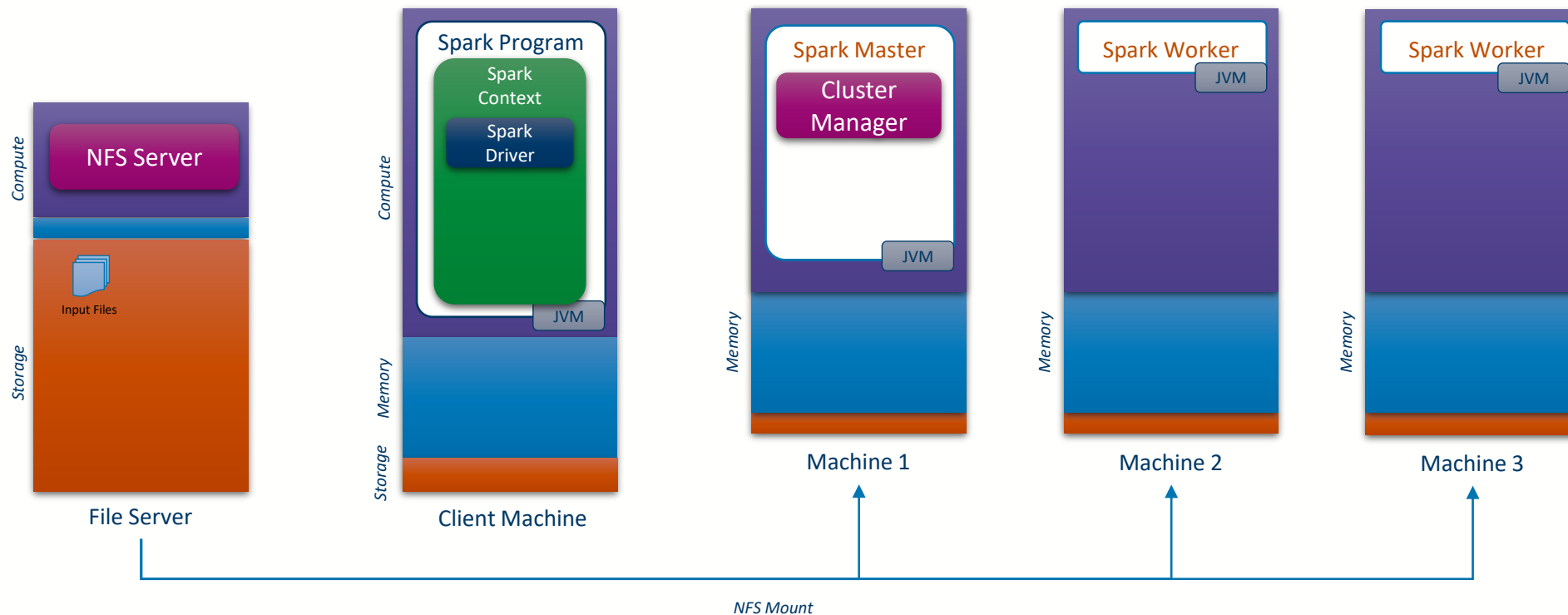
2 Create Spark Context





Job execution anatomy

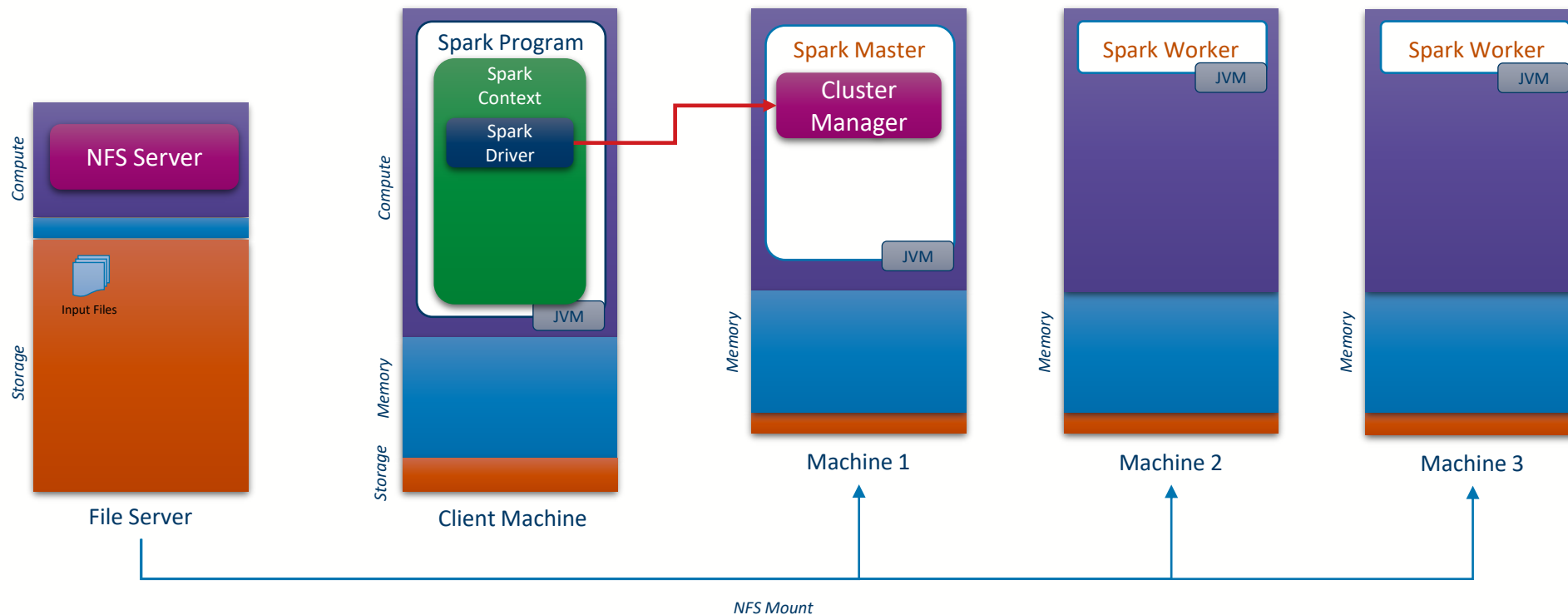
3 Init Spark Driver





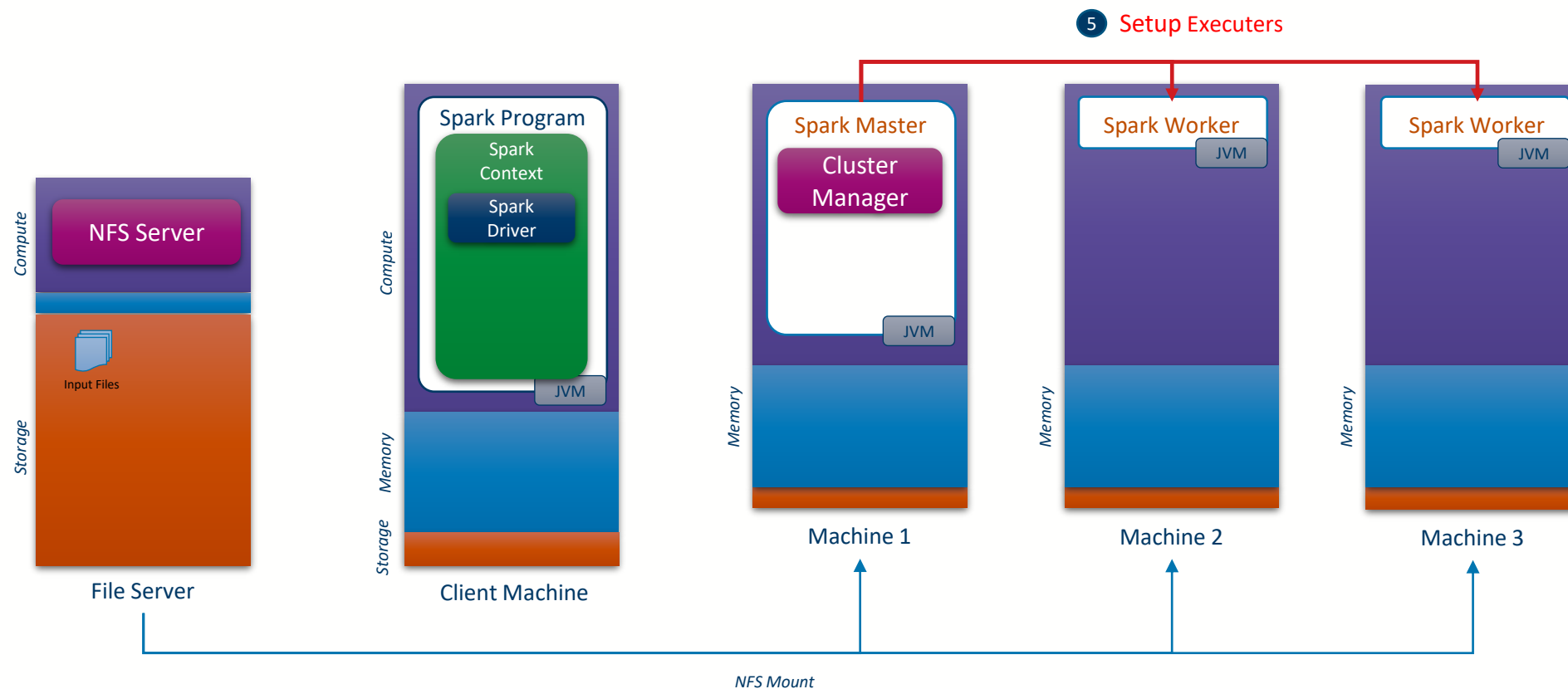
Job execution anatomy

4 Request Resources





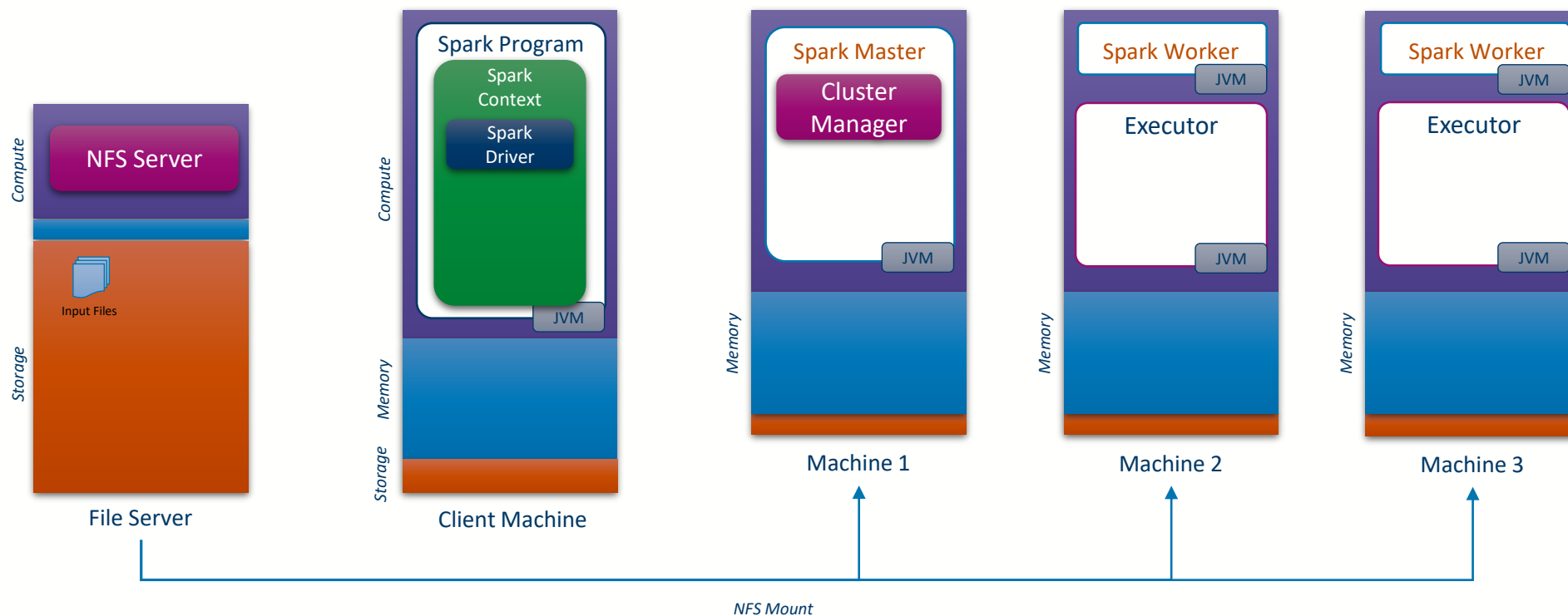
Job execution anatomy





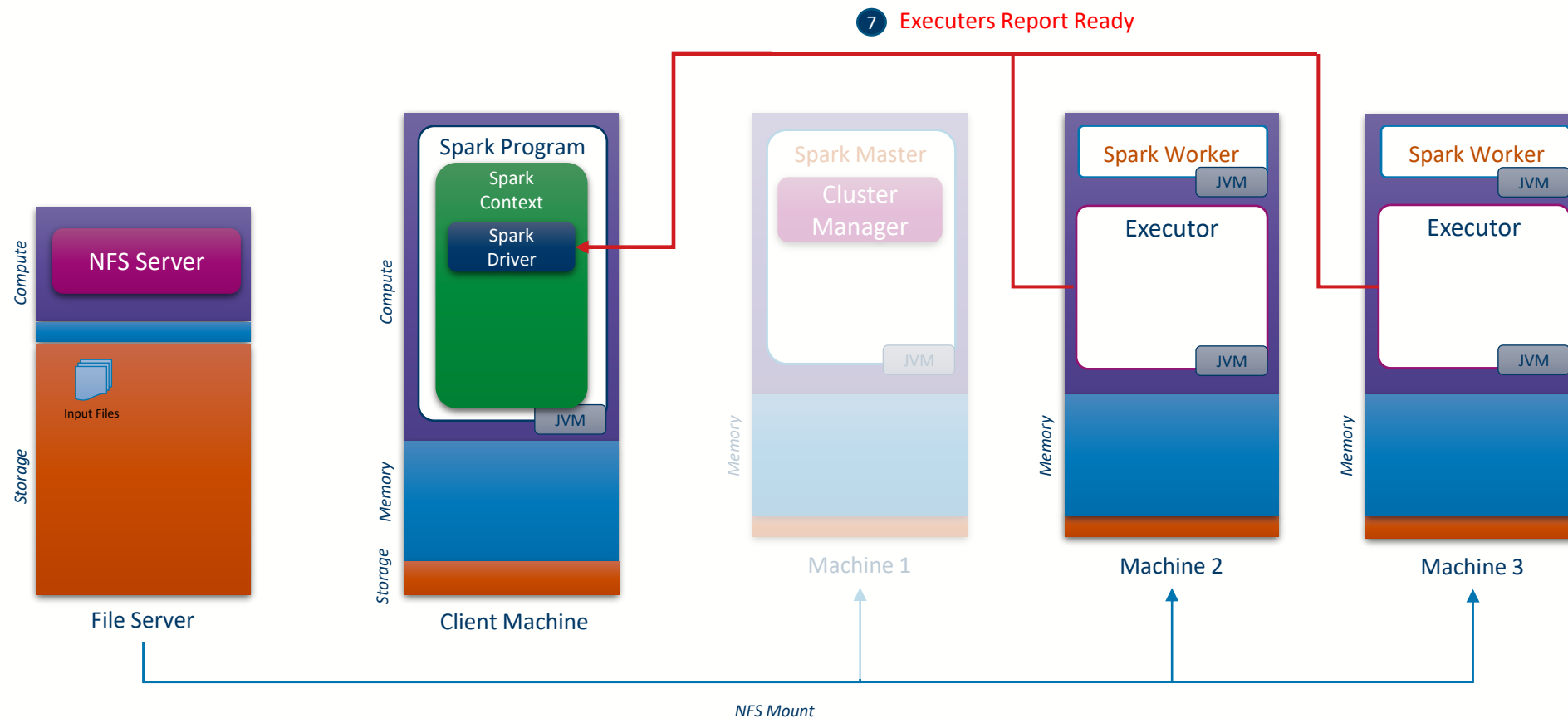
Job execution anatomy

6 Launch Executors





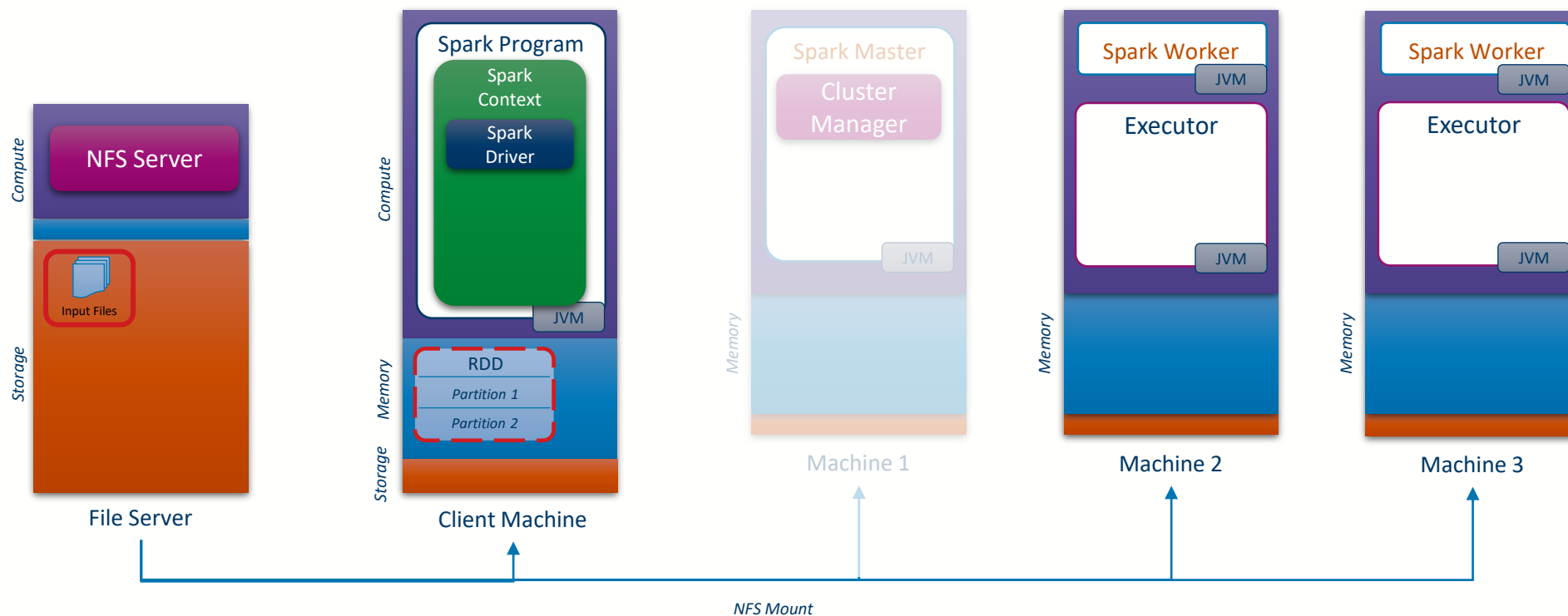
Job execution anatomy





Job execution anatomy

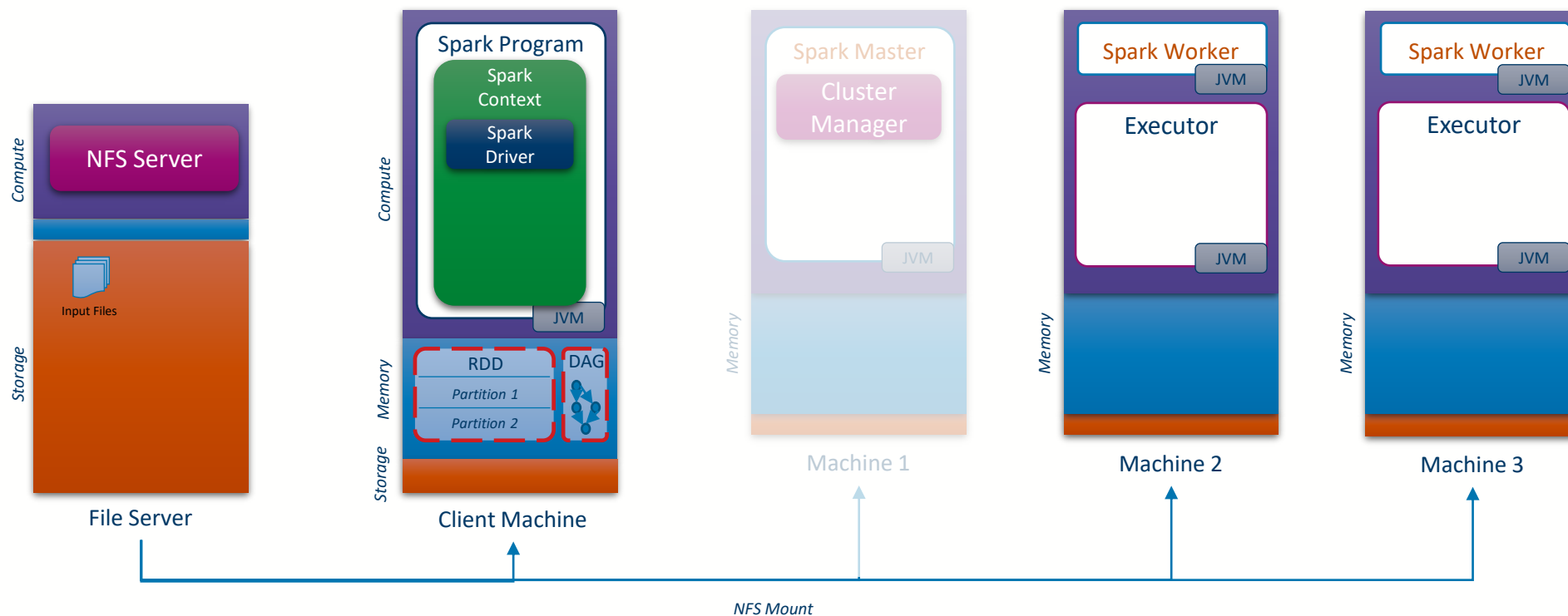
- 8 Create Input RDDs
(objects with partitions created via a data scan)





Job execution anatomy

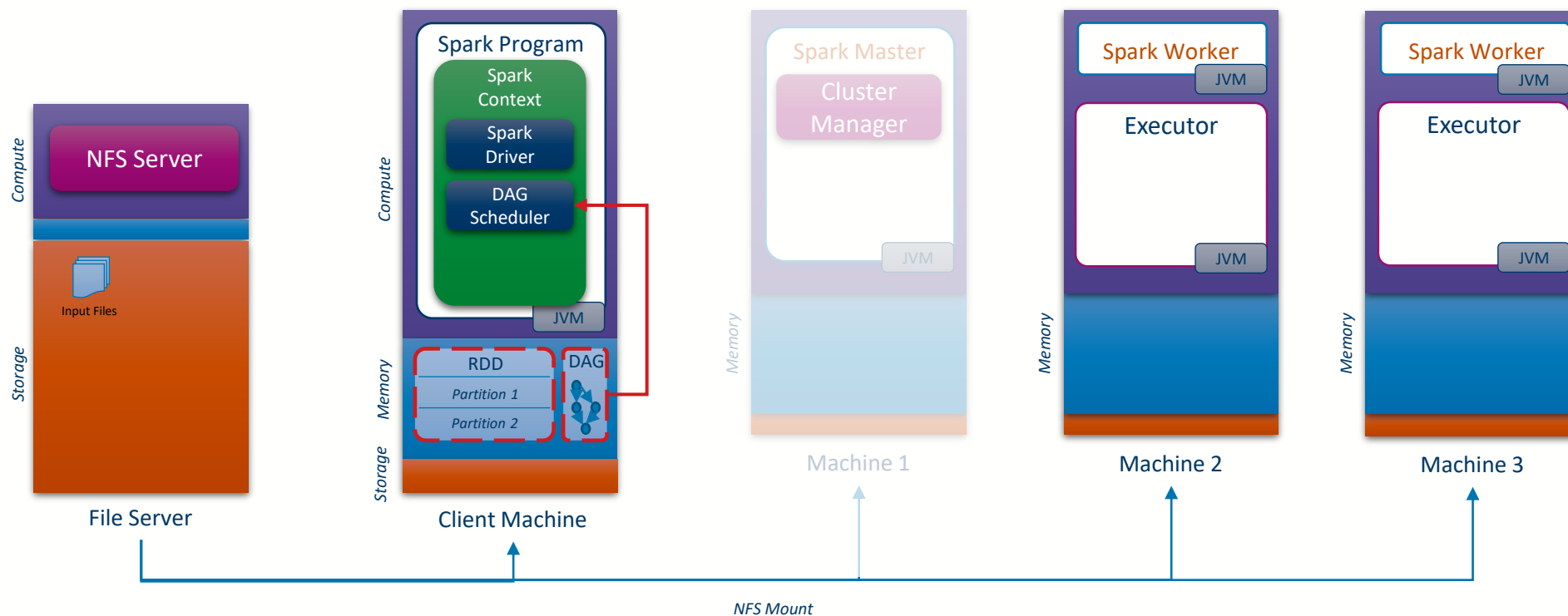
- 9 Execute Transformations in lazy mode,
Incrementally building the DAG





Job execution anatomy

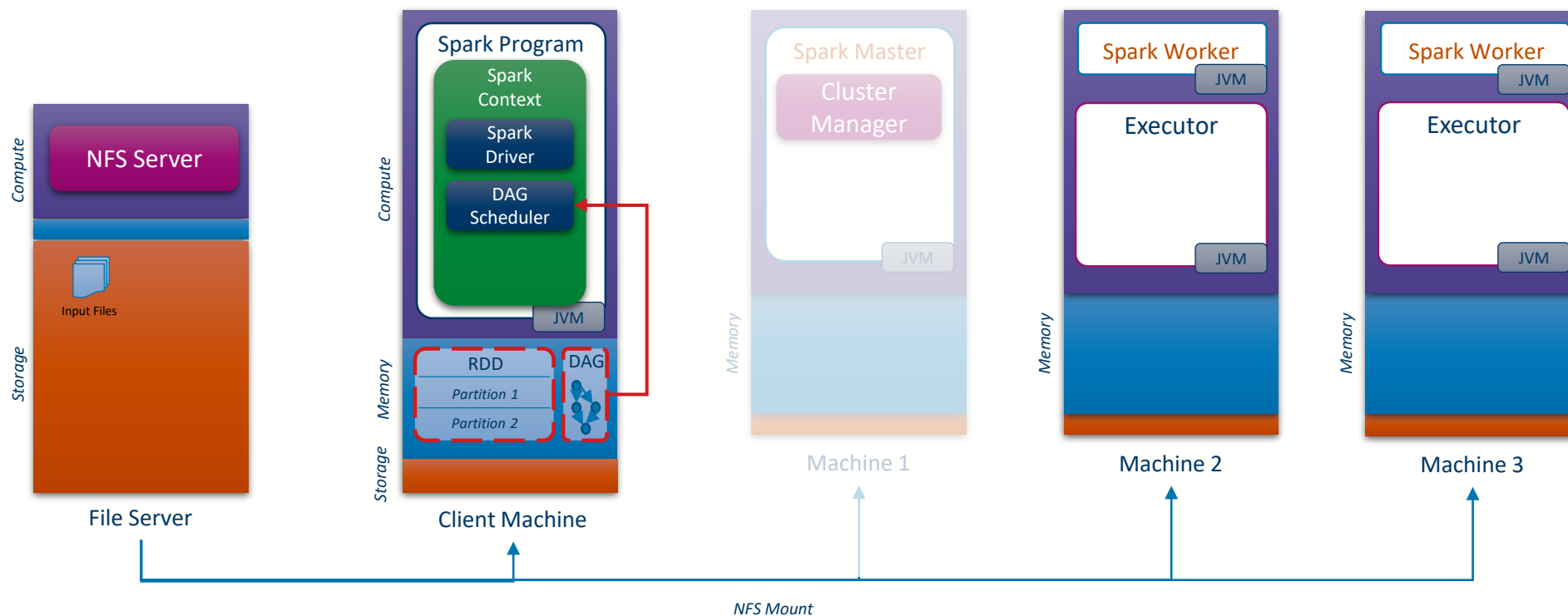
- 10 When an action is reached,
Send the DAG to the DAGScheduler





Job execution anatomy

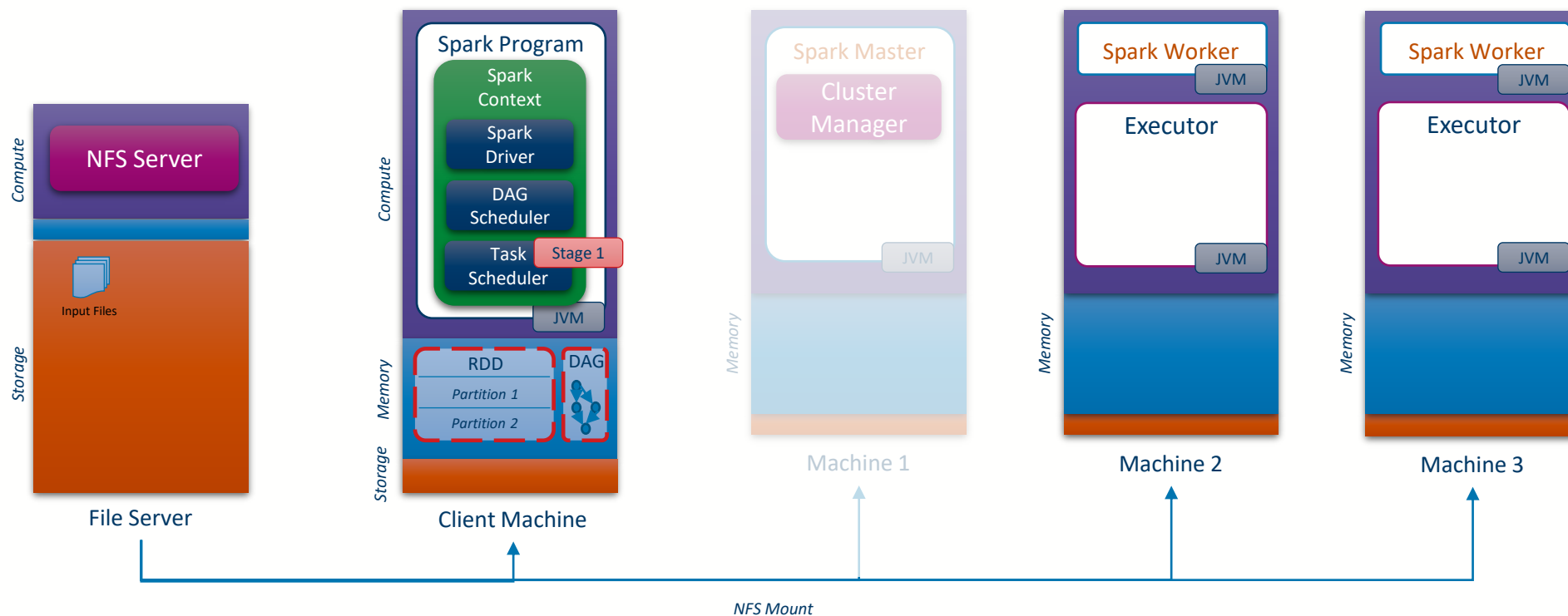
- 11 DAGScheduler creates the Execution plan (Tasks and Stages)





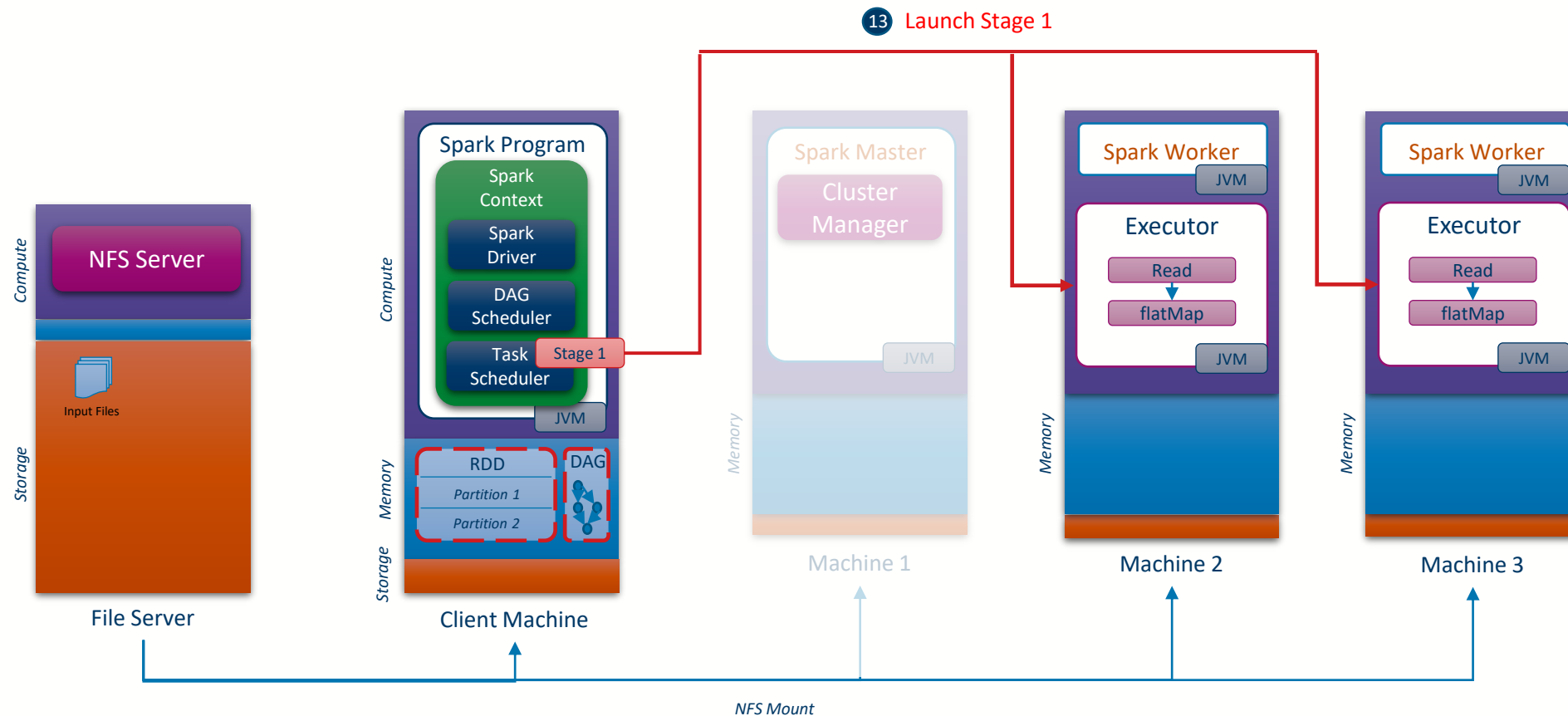
Job execution anatomy

- 12 Submit the first Stage to the Task Scheduler



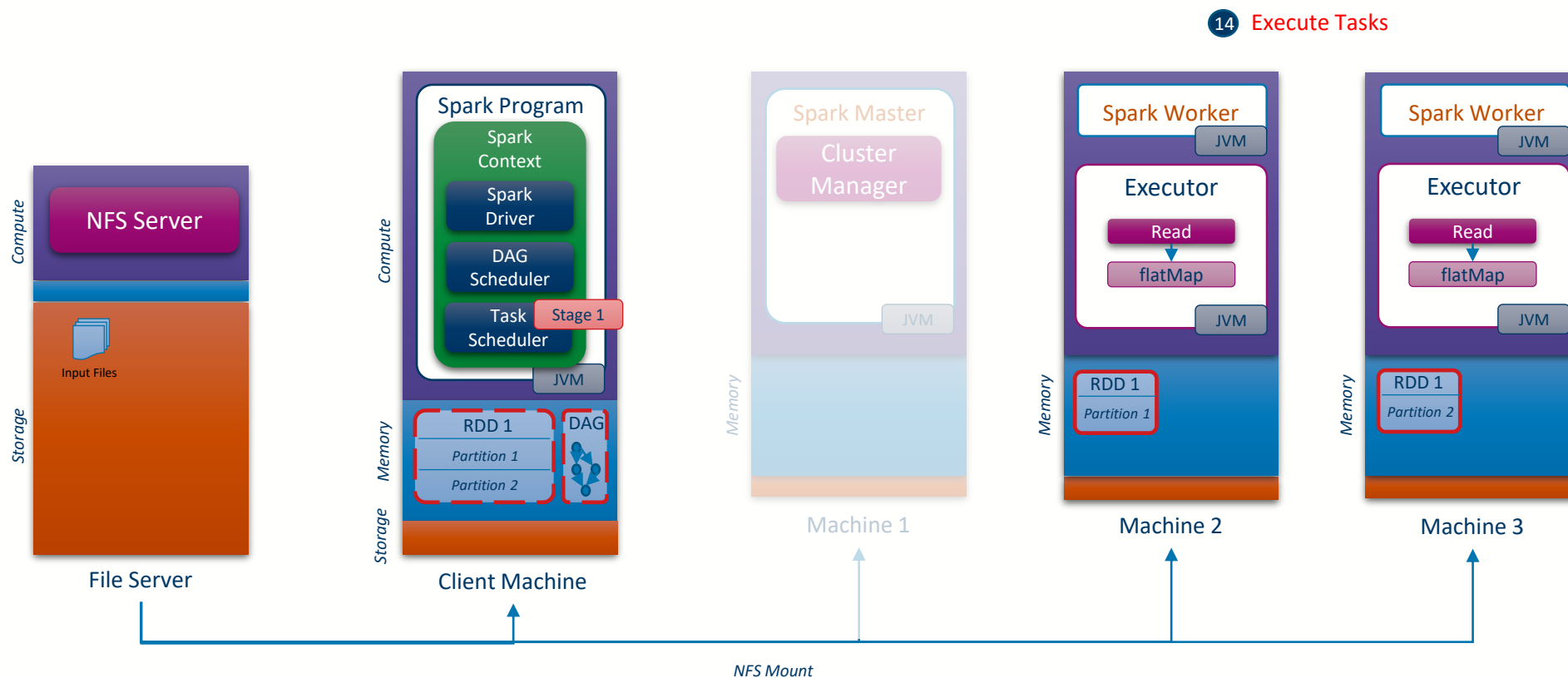


Job execution anatomy



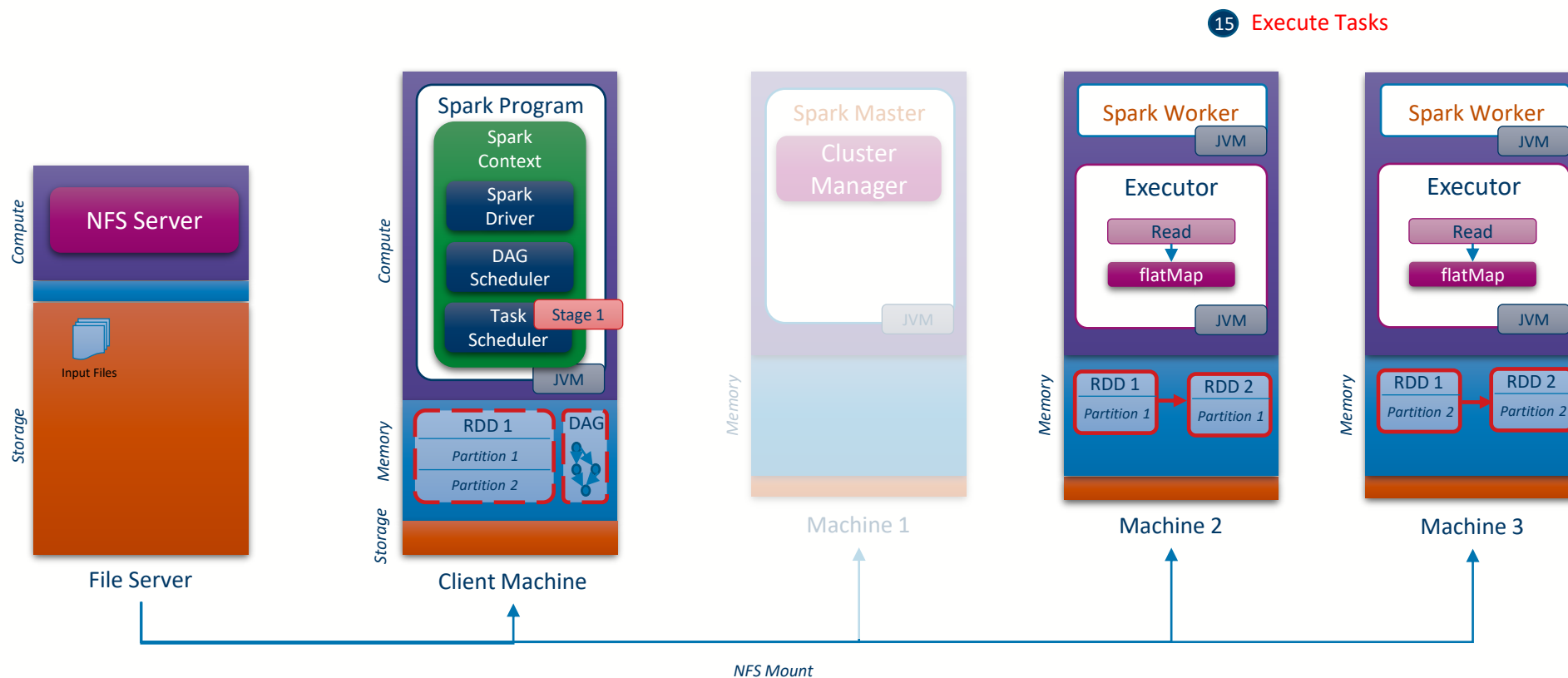


Job execution anatomy



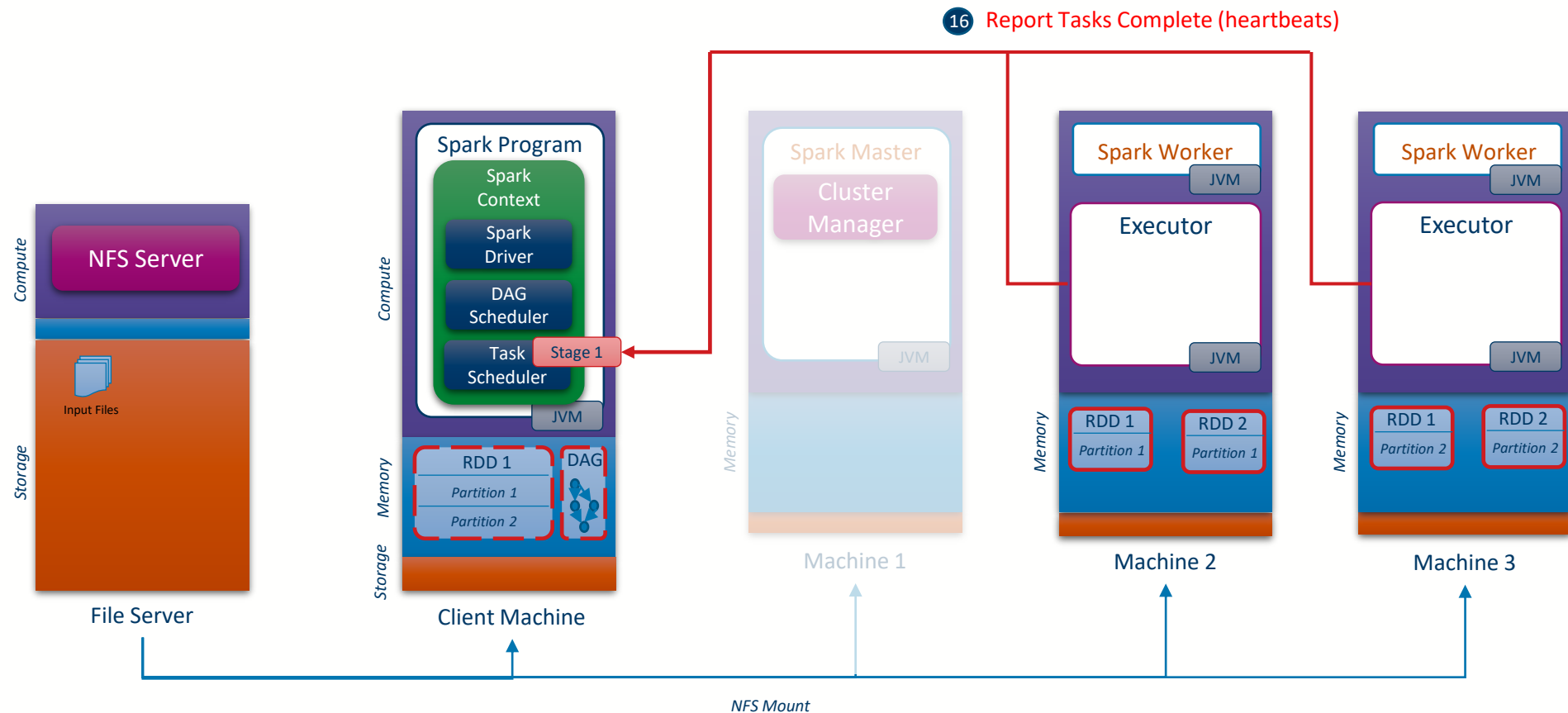


Job execution anatomy





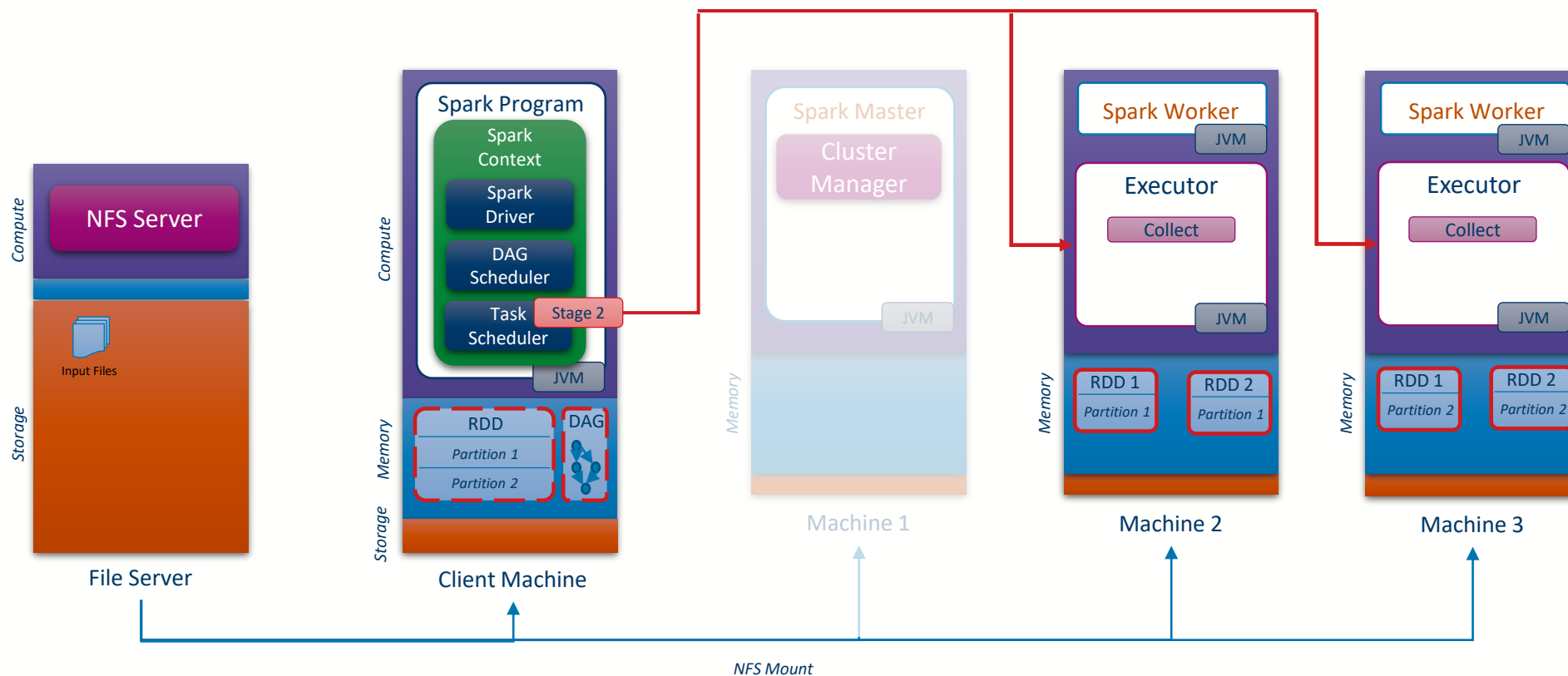
Job execution anatomy





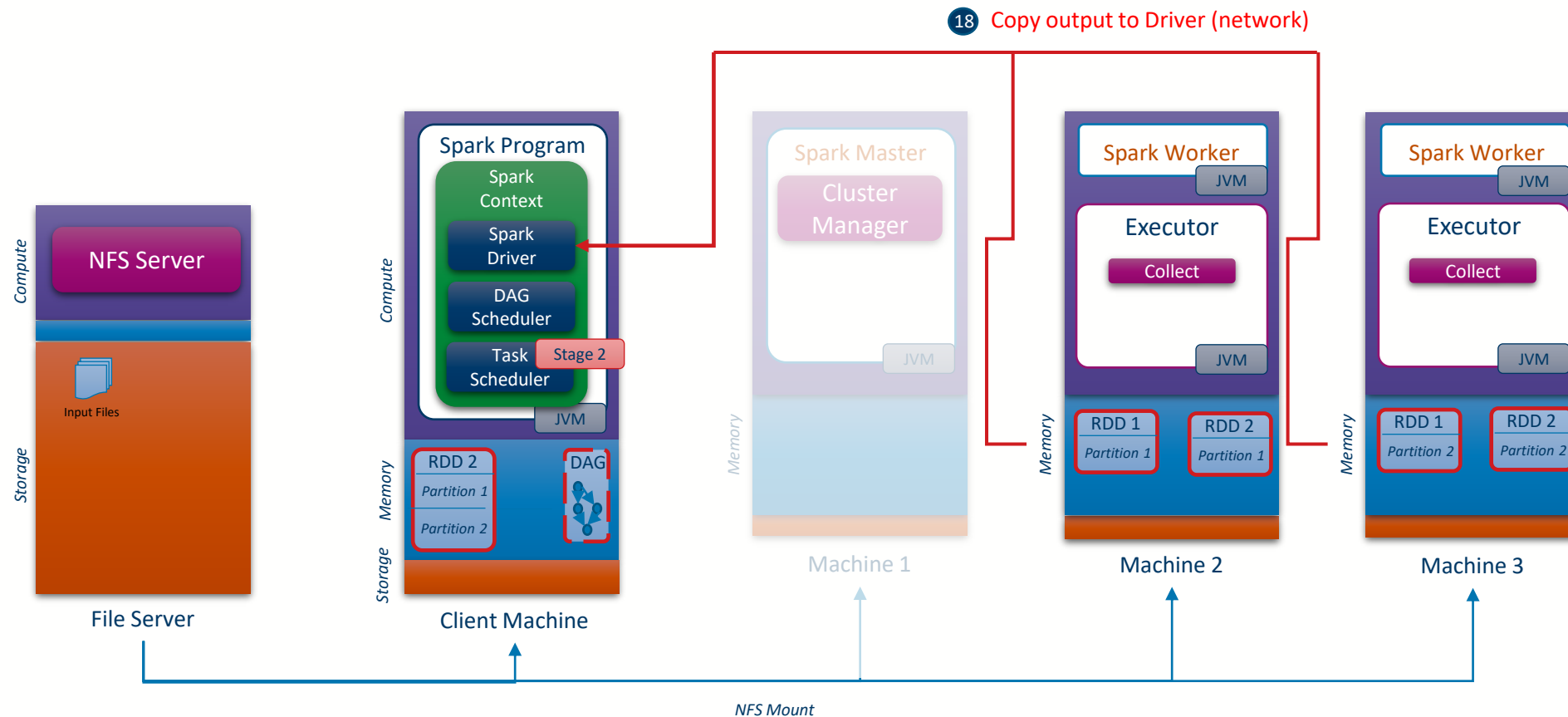
Job execution anatomy

17 Launch Stage 2



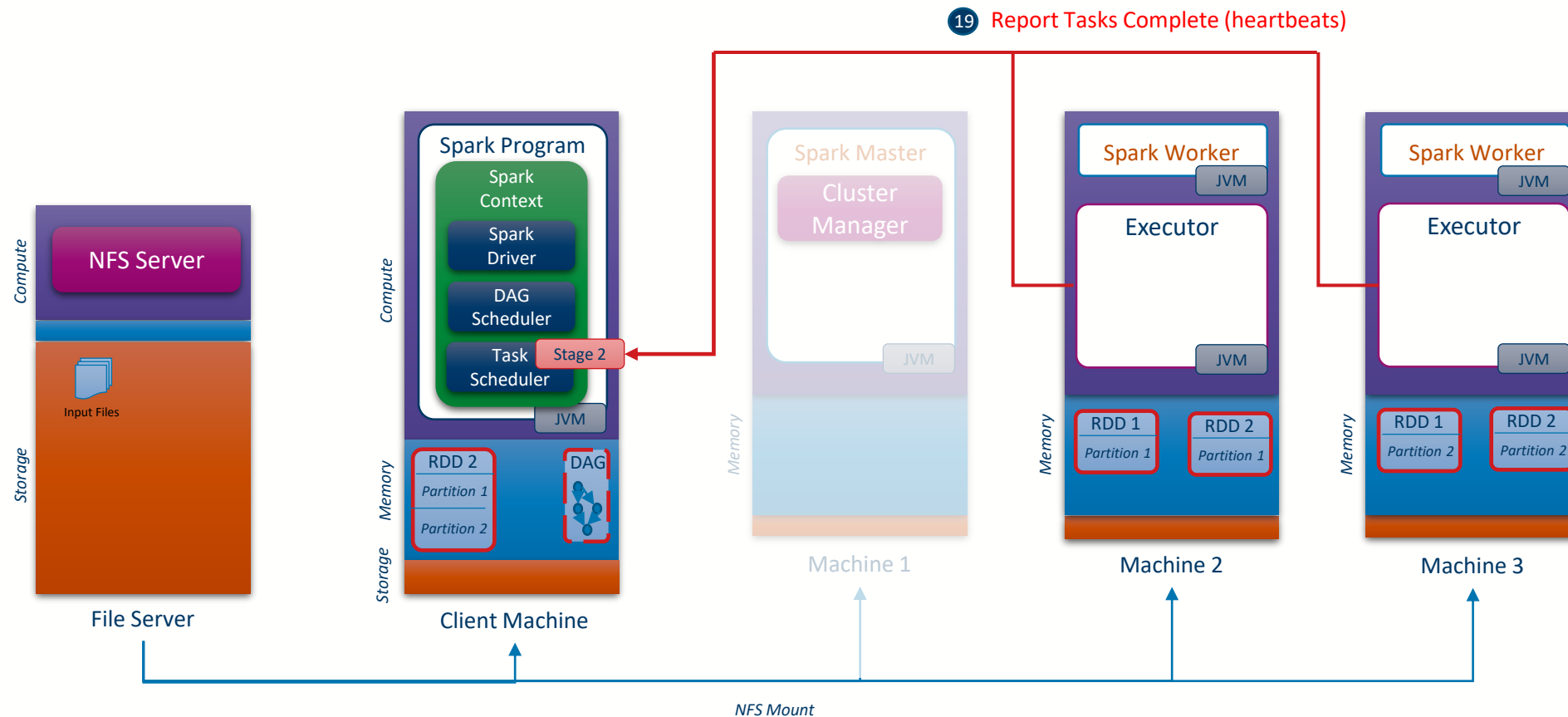


Job execution anatomy





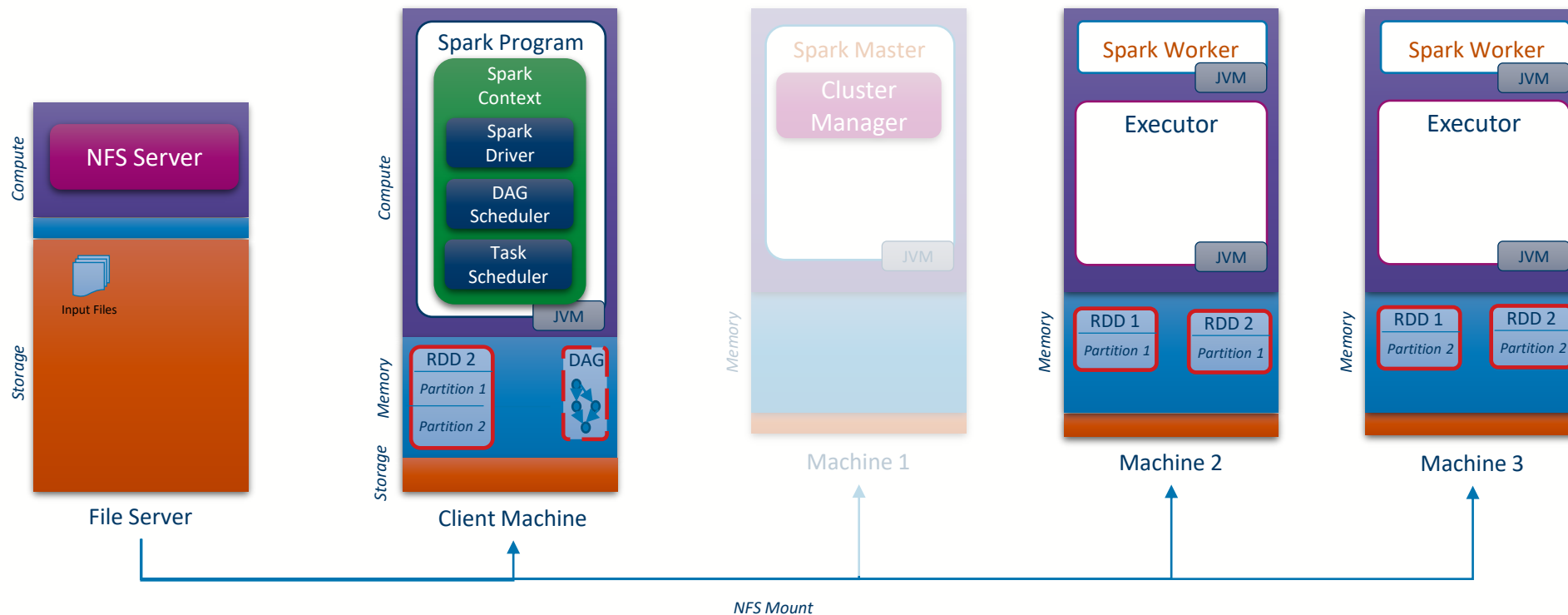
Job execution anatomy





Job execution anatomy

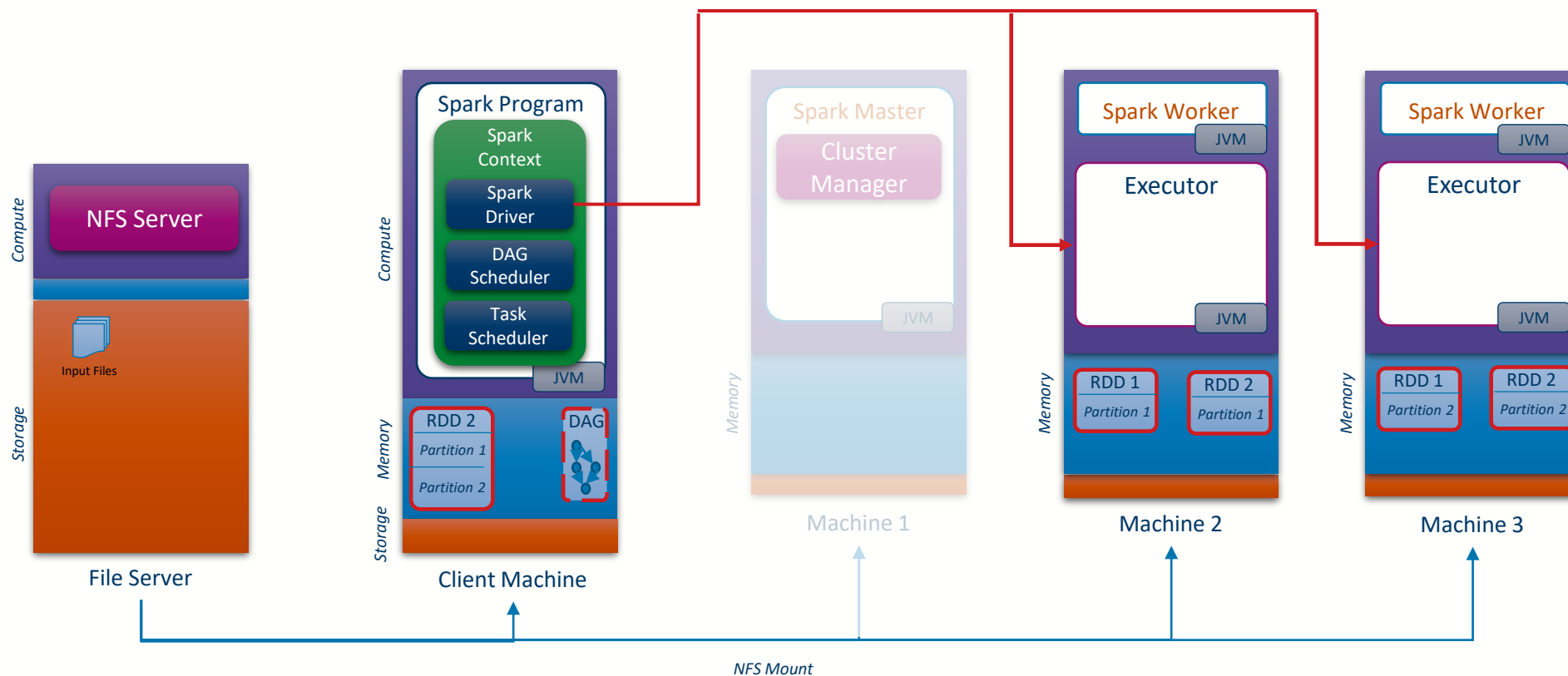
20 DAGScheduler Marks Job Complete





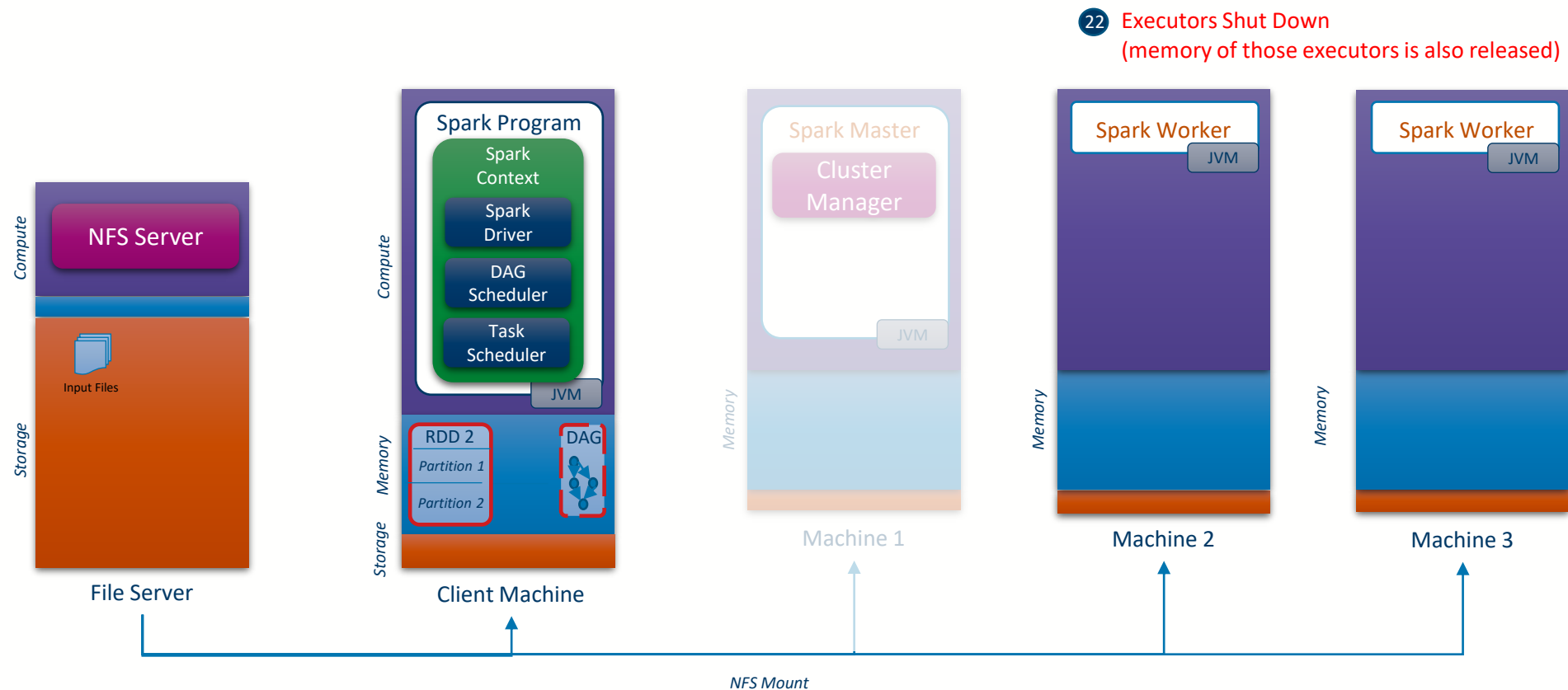
Job execution anatomy

21 Kill messages get sent to the Executors





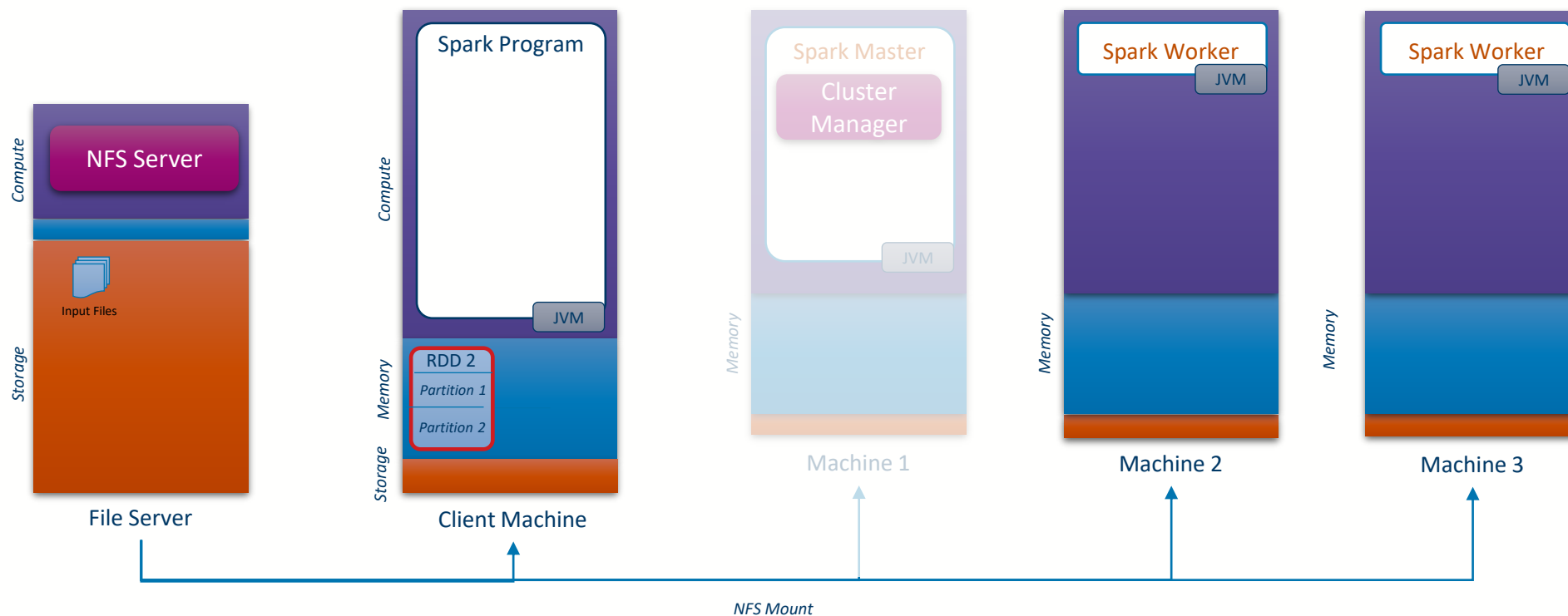
Job execution anatomy





Job execution anatomy

- 23 Job completes
(execute method returns in the main program)





University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow